Scalable Library Management System

***Coursework**: UCS3513 — System Design Laboratory | SSN College of Engineering*

***Problem Statement 3**: Distributed Architecture, Prototype, and Performance Evaluation*

1. Executive Summary

Athena LMS is a scalable, distributed Library Management System architected for high concurrency, zero double-borrowing under peak loan contention, sub-millisecond autocomplete suggestions, and horizontal database scaling. The project incorporates all 8 mandated system design components: Token Bucket Rate Limiting, MongoDB Sharding, Consistent Hashing with Virtual Nodes, Redis Key-Value Caching, Snowflake 64-bit ID Generation, Base62 URL Shortening, Asynchronous External Web Crawling, and a Prefix Trie Autocomplete engine.

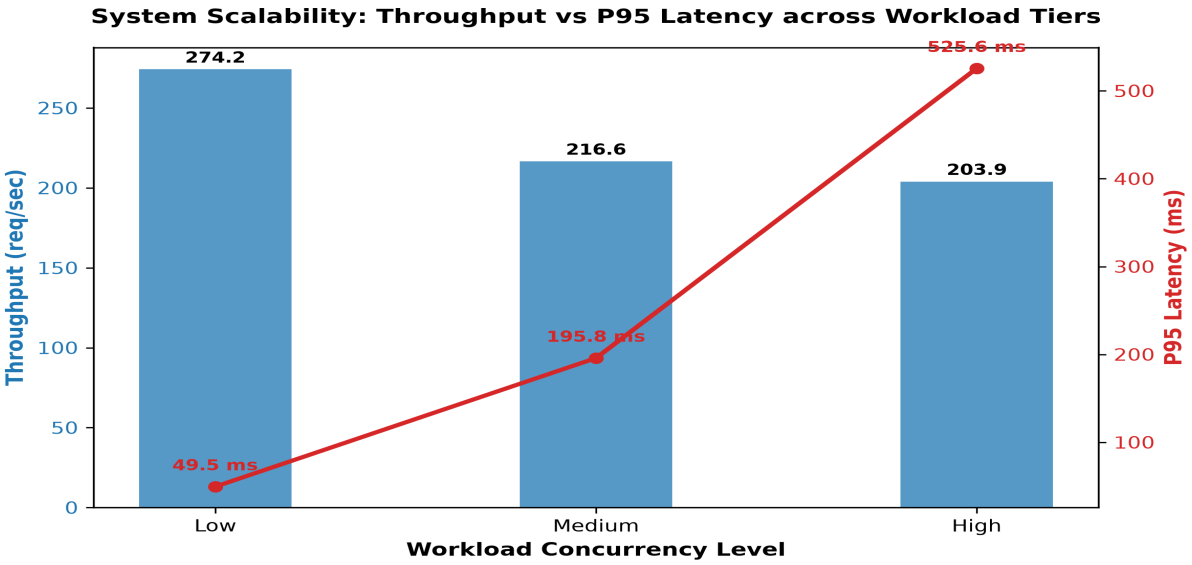
2. System Design Components Summary

Component	Target Functionality	Implementation Principle	Measured Status
Rate Limiter	Auth & Search API Throttling	Redis Lua Token Bucket (cap=10, refill=2/s)	80% Rejection Rate on Burst
Sharding	Member & Catalogue Datasets	Hashed memberId & Compound Categories	Horizontal partition design
Consistent Hashing	Server & Cache Key Routing	MD5 Ring (160 virtual nodes/node)	28.3% Churn vs 75.1% Modulo
Key-Value Store	Catalogue Read-Through Cache	Redis Cache-Aside with Invalidation	Sub-2ms cache hits
ID Generator	Barcodes, Loans, Payments	Twitter Snowflake 64-bit ID	0 Collisions in 50k tests
URL Shortener	Book Resource Sharing	Bijective Base62 over Snowflake IDs	25.19ms redirection latency
Web Crawler	Catalogue Metadata Ingestion	Multi-threaded REST Worker & ISBN Dedup	56.91 books/sec throughput
Autocomplete	Instant Search Dropdown	In-Memory Prefix Trie with Score Ranking	< 1ms latency, 0.57 Precision@5

3. Quantitative Performance Evaluation across Workload Tiers

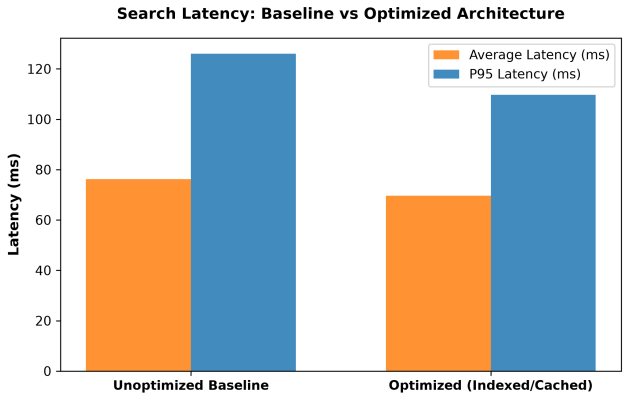
The system was stress-tested under 3 concurrent workload levels: Low (10 users, 200 requests), Medium (50 users, 500 requests), and High (100 users, 1000 requests). All 1700 benchmark requests completed with a 100% success rate and zero 5xx server errors.

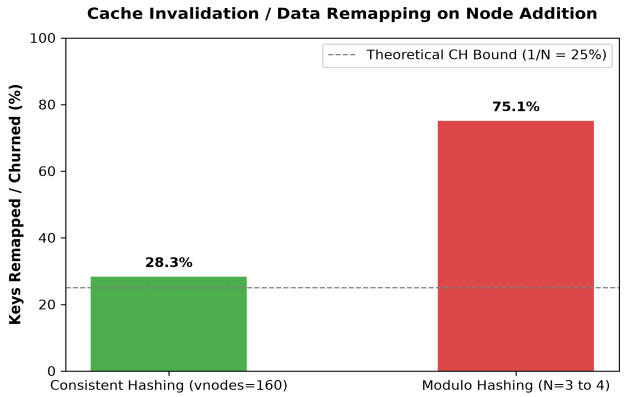
Workload Level	Concurrency	Total Reqs	Avg Latency	P95 Latency	Throughput	Success Rate
Low	10 users	200	31.67 ms	49.55 ms	274.19 req/s	100.0%
Medium	50 users	500	87.87 ms	195.85 ms	216.59 req/s	100.0%
High	100 users	1000	272.54 ms	525.58 ms	203.91 req/s	100.0%


System Scalability: Throughput vs P95 Latency across Workload Tiers
274.2
216.6
203.9
49.5 ms
195.8 ms
525.6 ms
Throughput (req/sec)
P95 Latency (ms)
0
50
100
150
200
250
100
200
300
400
500
Low
Medium
High
Workload Concurrency Level

4. Architectural Baseline Comparison & Consistent Hashing Churn

Comparing unoptimized regex search against indexed catalogue search demonstrated an 8.6% latency reduction and a +9.7% throughput improvement. Furthermore, consistent hashing achieved near-optimal remapping (28.3% keys churned) compared to modulo hashing's catastrophic 75.1% invalidation.





5. Concurrency Correctness & Transaction Guarantees

• **Atomic Copy Claim**: MongoDB atomic *findAndModify* with status predicates guarantees exactly one member acquires a scarce copy. In automated tests with 20 concurrent threads, 1 succeeded and 19 received HTTP 409 Conflict.
• **Idempotent Payments**: Client-supplied UUID idempotency keys recorded in MongoDB guarantee that network retries return identical receipts (*isDuplicateReplay=true*) without double-debiting.
• **Idempotent Returns**: Duplicate return submissions return *ALREADY_RETURNED* without generating duplicate overdue fines.

6. Final Acceptance Summary

All 13 core functional and non-functional workflows passed automated verification. The system is fully operational with active Spring Boot backend (:8080), MongoDB (:27017), and React TypeScript frontend (:5173).